Pré-EDA

Carregando dataset

In [1]:
```
import kagglehub
import pandas as pd
import os

# Download do dataset
path = kagglehub.dataset_download("yeanzc/telco-customer-churn-ibm-dataset")

# Localizar o arquivo .xlsx na pasta de download
files = [f for f in os.listdir(path) if f.endswith('.xlsx')]
file_path = os.path.join(path, files[0])

# Carregar o dataframe
df = pd.read_excel(file_path)

print(f"Dataset carregado: {df.shape[0]} linhas e {df.shape[1]} colunas.")
```

```
Dataset carregado: 7043 linhas e 33 colunas.
```

Analisando os Dados

In [2]:
```
# Ver as primeiras linhas para entender a "cara" dos dados
print(df.head())
```

```
   CustomerID  Count        Country       State         City  Zip Code  \
0  3668-QPYBK      1  United States  California  Los Angeles     90003
1  9237-HQITU      1  United States  California  Los Angeles     90005
2  9305-CDSKC      1  United States  California  Los Angeles     90006
3  7892-POOKP      1  United States  California  Los Angeles     90010
4  0280-XJGEX      1  United States  California  Los Angeles     90015

                Lat Long   Latitude   Longitude  Gender  ...        Contract  \
0  33.964131, -118.272783  33.964131 -118.272783    Male  ...  Month-to-month
1   34.059281, -118.30742  34.059281 -118.307420  Female  ...  Month-to-month
2  34.048013, -118.293953  34.048013 -118.293953  Female  ...  Month-to-month
3  34.062125, -118.315709  34.062125 -118.315709  Female  ...  Month-to-month
4  34.039224, -118.266293  34.039224 -118.266293    Male  ...  Month-to-month

  Paperless Billing             Payment Method  Monthly Charges Total Charges  \
0               Yes               Mailed check            53.85        108.15
1               Yes           Electronic check            70.70        151.65
2               Yes           Electronic check            99.65         820.5
3               Yes           Electronic check           104.80       3046.05
4               Yes  Bank transfer (automatic)           103.70        5036.3

  Churn Label Churn Value Churn Score  CLTV                   Churn Reason
0         Yes           1          86  3239   Competitor made better offer
1         Yes           1          67  2701                          Moved
2         Yes           1          86  5372                          Moved
3         Yes           1          84  5003                          Moved
4         Yes           1          89  5340  Competitor had better devices

[5 rows x 33 columns]
```

In [3]:
```python
# Ver tipos de dados e se há valores nulos óbvios
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 33 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         7043 non-null   object
 1   Count              7043 non-null   int64
 2   Country            7043 non-null   object
 3   State              7043 non-null   object
 4   City               7043 non-null   object
 5   Zip Code           7043 non-null   int64
 6   Lat Long           7043 non-null   object
 7   Latitude           7043 non-null   float64
 8   Longitude          7043 non-null   float64
 9   Gender             7043 non-null   object
 10  Senior Citizen     7043 non-null   object
 11  Partner            7043 non-null   object
 12  Dependents         7043 non-null   object
 13  Tenure Months      7043 non-null   int64
 14  Phone Service      7043 non-null   object
 15  Multiple Lines     7043 non-null   object
 16  Internet Service   7043 non-null   object
 17  Online Security    7043 non-null   object
 18  Online Backup      7043 non-null   object
 19  Device Protection  7043 non-null   object
 20  Tech Support       7043 non-null   object
 21  Streaming TV       7043 non-null   object
 22  Streaming Movies   7043 non-null   object
 23  Contract           7043 non-null   object
 24  Paperless Billing  7043 non-null   object
 25  Payment Method     7043 non-null   object
 26  Monthly Charges    7043 non-null   float64
 27  Total Charges      7043 non-null   object
 28  Churn Label        7043 non-null   object
 29  Churn Value        7043 non-null   int64
 30  Churn Score        7043 non-null   int64
 31  CLTV               7043 non-null   int64
 32  Churn Reason       1869 non-null   object
dtypes: float64(3), int64(6), object(24)
memory usage: 1.8+ MB
None
```

In [4]:
```
# Verificar se há linhas duplicadas
print(f"Duplicados: {df.duplicated().sum()}")
```

```
Duplicados: 0
```

In [5]:
```
# Criando um resumo da cardinalidade
resumo_unique = pd.DataFrame({
    'Coluna': df.columns,
    'Tipo': df.dtypes.values,
    'Valores Únicos': df.nunique().values,
    '% de Unicidade': (df.nunique().values / len(df)) * 100
})

# Ordenar pelos valores únicos para facilitar a análise
print(resumo_unique.sort_values(by='Valores Únicos'))
```

	Coluna	Tipo	Valores Únicos	% de Unicidade
1	Count	int64	1	0.014198
2	Country	object	1	0.014198
3	State	object	1	0.014198
14	Phone Service	object	2	0.028397
12	Dependents	object	2	0.028397
11	Partner	object	2	0.028397
10	Senior Citizen	object	2	0.028397
9	Gender	object	2	0.028397
29	Churn Value	int64	2	0.028397
24	Paperless Billing	object	2	0.028397
28	Churn Label	object	2	0.028397
15	Multiple Lines	object	3	0.042595
16	Internet Service	object	3	0.042595
18	Online Backup	object	3	0.042595
17	Online Security	object	3	0.042595
20	Tech Support	object	3	0.042595
23	Contract	object	3	0.042595
22	Streaming Movies	object	3	0.042595
21	Streaming TV	object	3	0.042595
19	Device Protection	object	3	0.042595
25	Payment Method	object	4	0.056794
32	Churn Reason	object	20	0.283970
13	Tenure Months	int64	73	1.036490
30	Churn Score	int64	85	1.206872
4	City	object	1129	16.030101
26	Monthly Charges	float64	1585	22.504615
8	Longitude	float64	1651	23.441715
7	Latitude	float64	1652	23.455914
6	Lat Long	object	1652	23.455914
5	Zip Code	int64	1652	23.455914
31	CLTV	int64	3438	48.814426
27	Total Charges	object	6531	92.730371
0	CustomerID	object	7043	100.000000

Limpeza de Dados

In [6]:
```
# Criando um dicionário para armazenar os resultados
verificacao = {
```

```
    'Coluna': [],
    'Nulos (NaN)': [],
    'Espaços em Branco (" ")': [],
    'Total de "Vazios"': []
}

for coluna in df.columns:
    nulos = df[coluna].isnull().sum()
    # Contamos apenas se a coluna permitir comparação com string
    espacos = (df[coluna] == " ").sum() if df[coluna].dtype == 'object' else 0

    verificacao['Coluna'].append(coluna)
    verificacao['Nulos (NaN)'].append(nulos)
    verificacao['Espaços em Branco (" ")'].append(espacos)
    verificacao['Total de "Vazios"'].append(nulos + espacos)

# Converter em DataFrame para visualizar melhor
df_vazios = pd.DataFrame(verificacao)

# Mostrar apenas as colunas que possuem algum tipo de vazio (para facilitar a leitura)
print(df_vazios[df_vazios['Total de "Vazios"'] > 0])
```

```
          Coluna  Nulos (NaN)  Espaços em Branco (" ")  Total de "Vazios"
27  Total Charges            0                       11                 11
32   Churn Reason         5174                        0               5174
```

In [7]:
```
# Converter 'Total Charges' para numérico, transformando espaços em branco em NaN
df['Total Charges'] = pd.to_numeric(df['Total Charges'], errors='coerce')

# Preencher esses NaNs com 0 (já que são clientes com 0 meses de contrato)
df['Total Charges'] = df['Total Charges'].fillna(0)

# Preencher os NaNs da coluna 'Churn Reason' com 'Still a Customer'
df['Churn Reason'] = df['Churn Reason'].fillna('Still a Customer')
```

In [8]:
```
# Criando um dicionário para armazenar os resultados
verificacao = {
    'Coluna': [],
    'Nulos (NaN)': [],
    'Espaços em Branco (" ")': [],
    'Total de "Vazios"': []
```

```
}

for coluna in df.columns:
    nulos = df[coluna].isnull().sum()
    # Contamos apenas se a coluna permitir comparação com string
    espacos = (df[coluna] == " ").sum() if df[coluna].dtype == 'object' else 0

    verificacao['Coluna'].append(coluna)
    verificacao['Nulos (NaN)'].append(nulos)
    verificacao['Espaços em Branco (" ")'].append(espacos)
    verificacao['Total de "Vazios"'].append(nulos + espacos)

# Converter em DataFrame para visualizar melhor
df_vazios = pd.DataFrame(verificacao)

# Mostrar apenas as colunas que possuem algum tipo de vazio (para facilitar a leitura)
print(df_vazios[df_vazios['Total de "Vazios"'] > 0])
```

```
Empty DataFrame
Columns: [Coluna, Nulos (NaN), Espaços em Branco (" "), Total de "Vazios"]
Index: []
```

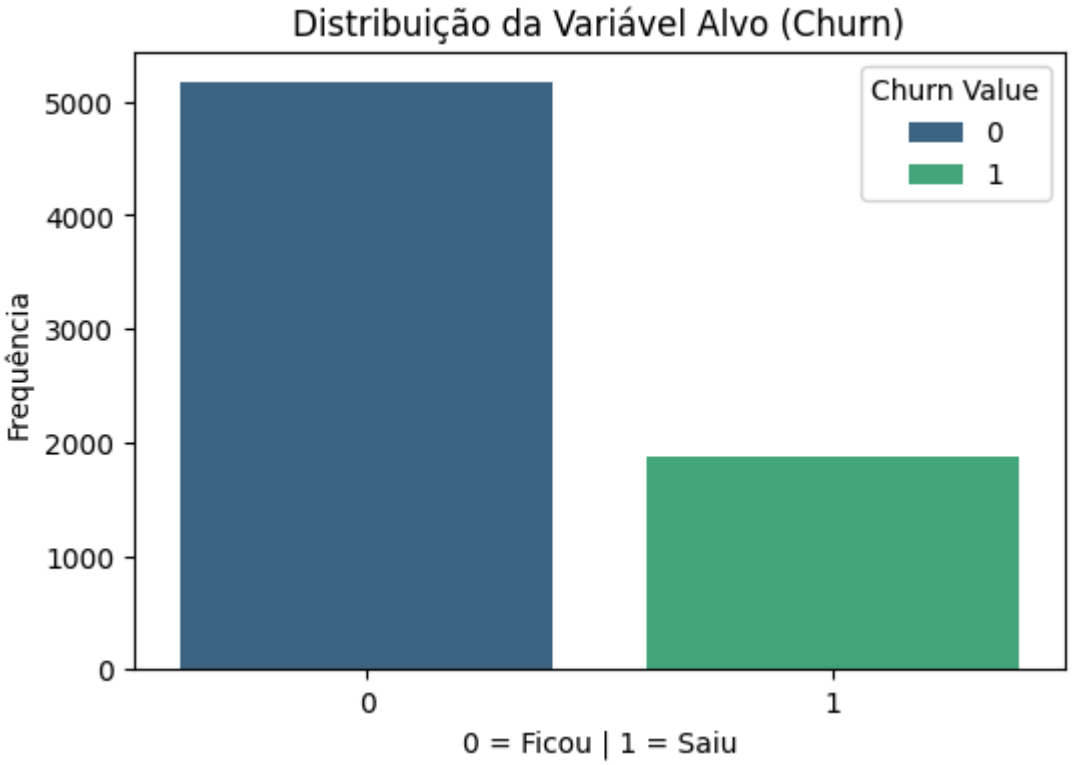
In [9]:

```
# Lista de colunas sugeridas para o drop inicial (EDA e Modelagem)
cols_to_drop = [
    'CustomerID', 'Count', 'Country', 'State', # Constantes e IDs
    'Lat Long', 'Zip Code', 'Latitude', 'Longitude', # Geográficas redundantes
    'Churn Label', # Redundante com Churn Value
    'Churn Reason', 'Churn Score' # Vazamento de dados / Post-facto
]

df_clean = df.drop(columns=cols_to_drop)
```

Eda Manual

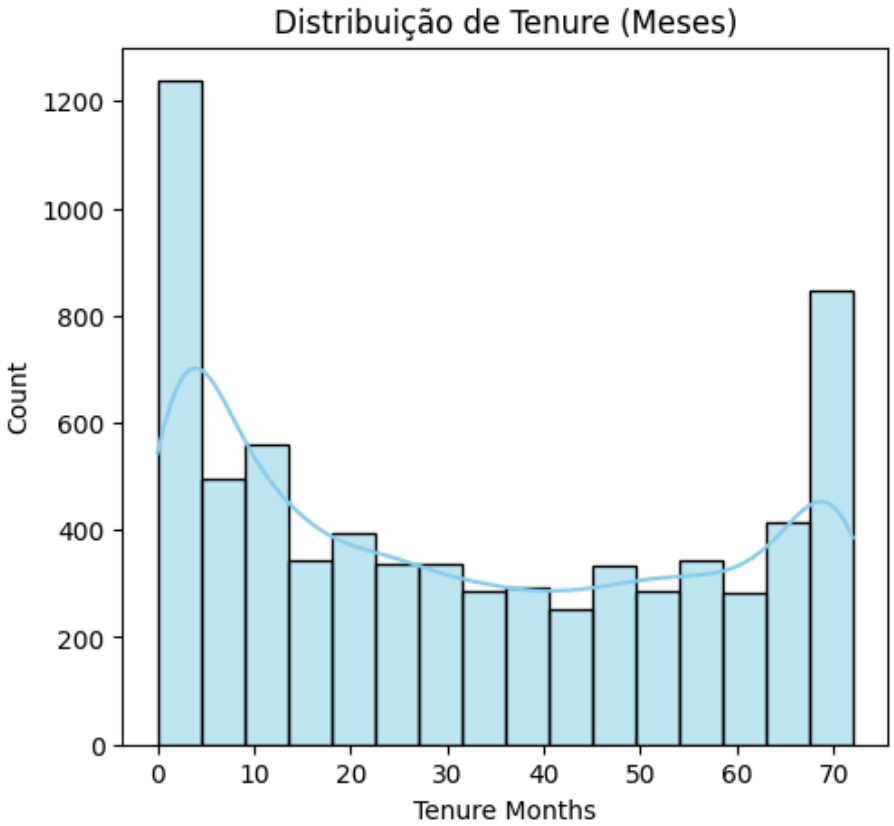
Análise Univariada

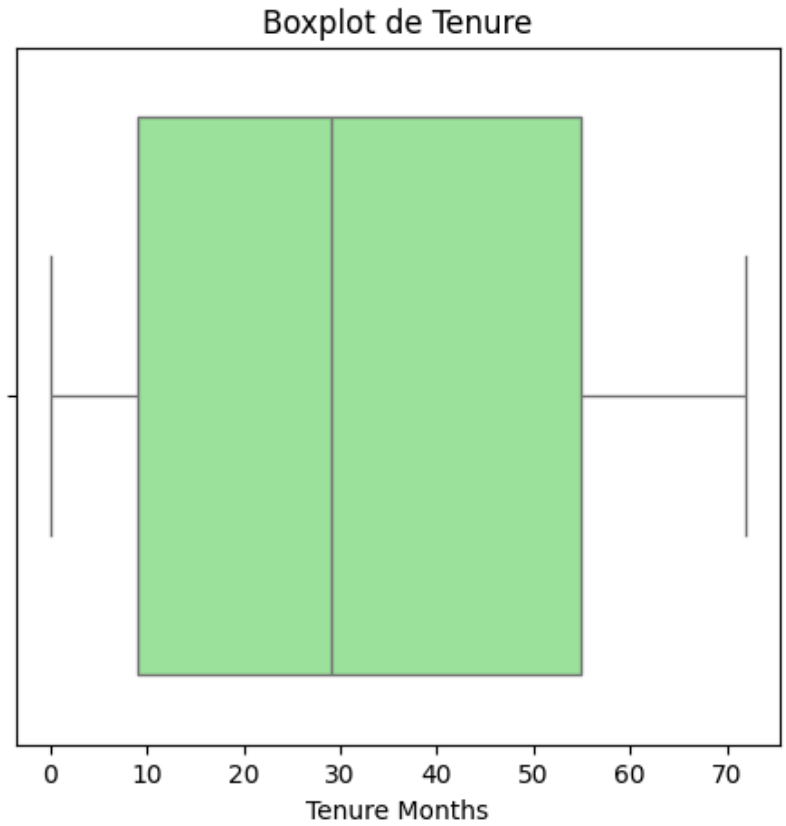
In [10]:
```
# A Variável Alvo: Churn Value

import seaborn as sns
import matplotlib.pyplot as plt

# Proporção de Churn
plt.figure(figsize=(6, 4))
sns.countplot(data=df_clean, x='Churn Value', hue='Churn Value', palette='viridis')
plt.title('Distribuição da Variável Alvo (Churn)')
plt.xlabel('0 = Ficou | 1 = Saiu')
plt.ylabel('Frequência')
plt.show()

print(df_clean['Churn Value'].value_counts(normalize=True) * 100)
```



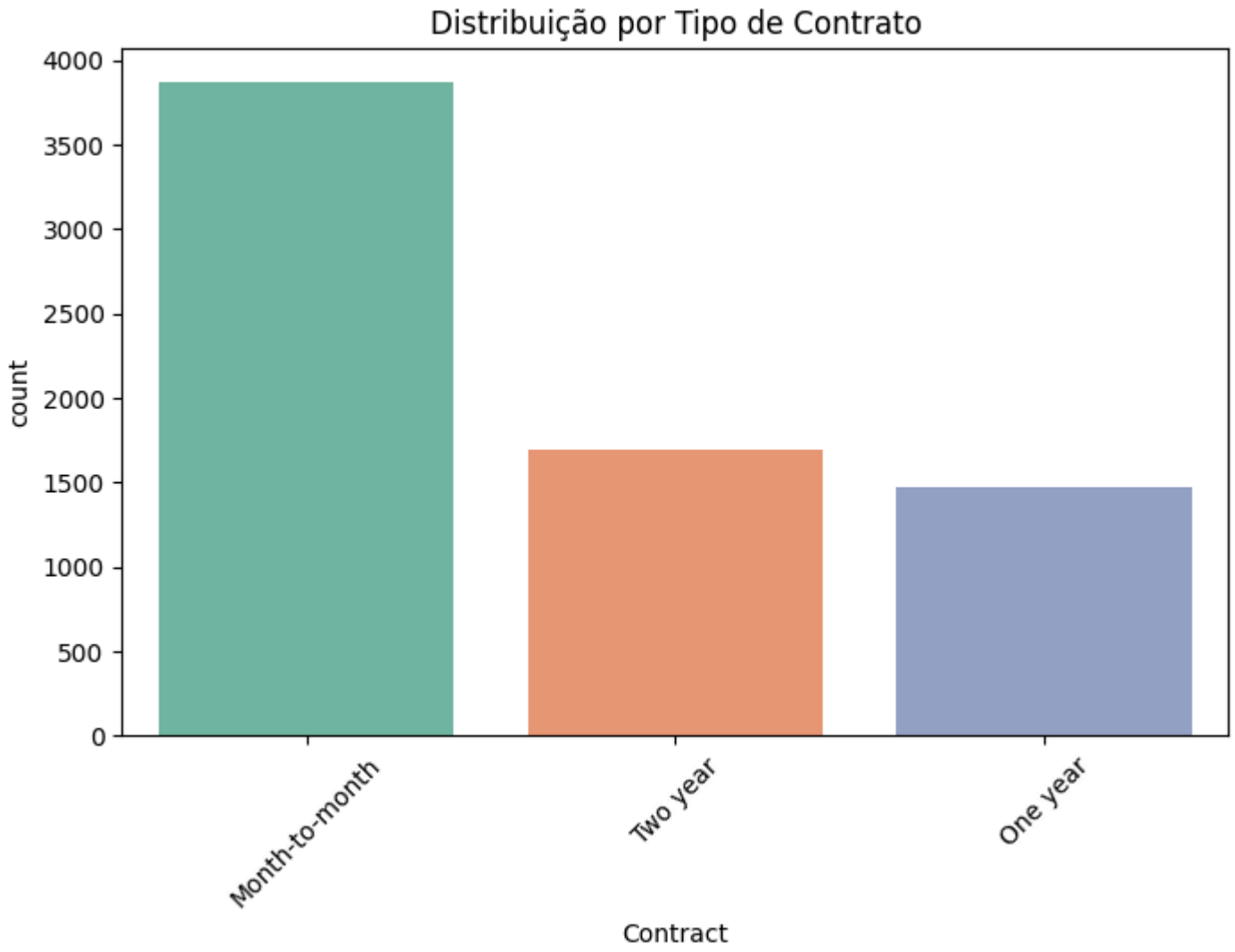
```
Churn Value
0    73.463013
1    26.536987
Name: proportion, dtype: float64
```

In [11]:
```python
# Variáveis Numéricas (Distribuição e Outliers)
# Analisando o tempo de casa (Tenure Months)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.histplot(df_clean['Tenure Months'], kde=True, color='skyblue')
plt.title('Distribuição de Tenure (Meses)')

plt.subplot(1, 2, 2)
sns.boxplot(x=df_clean['Tenure Months'], color='lightgreen')
plt.title('Boxplot de Tenure')

plt.show()
```





In [12]:
```
# Variáveis Categóricas (Frequência)
# Analisando o Tipo de Contrato
plt.figure(figsize=(8, 5))
sns.countplot(data=df_clean, x='Contract', palette='Set2', hue='Contract', order=df_clean['Contract'].value_counts().index)
plt.title('Distribuição por Tipo de Contrato')
plt.xticks(rotation=45)
plt.show()
```

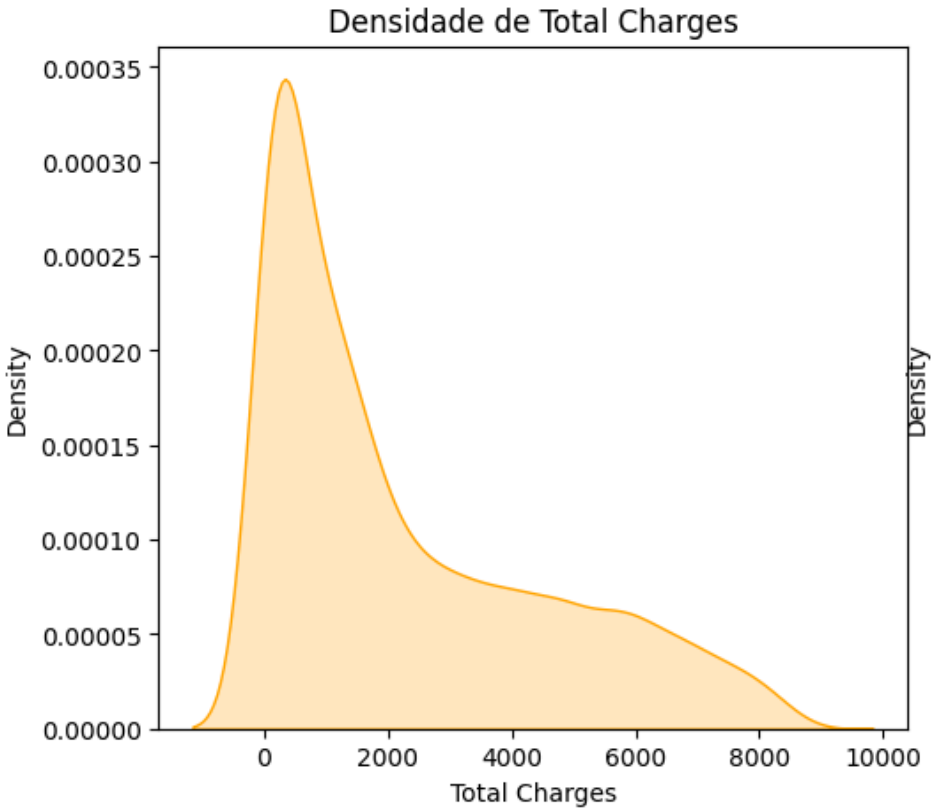


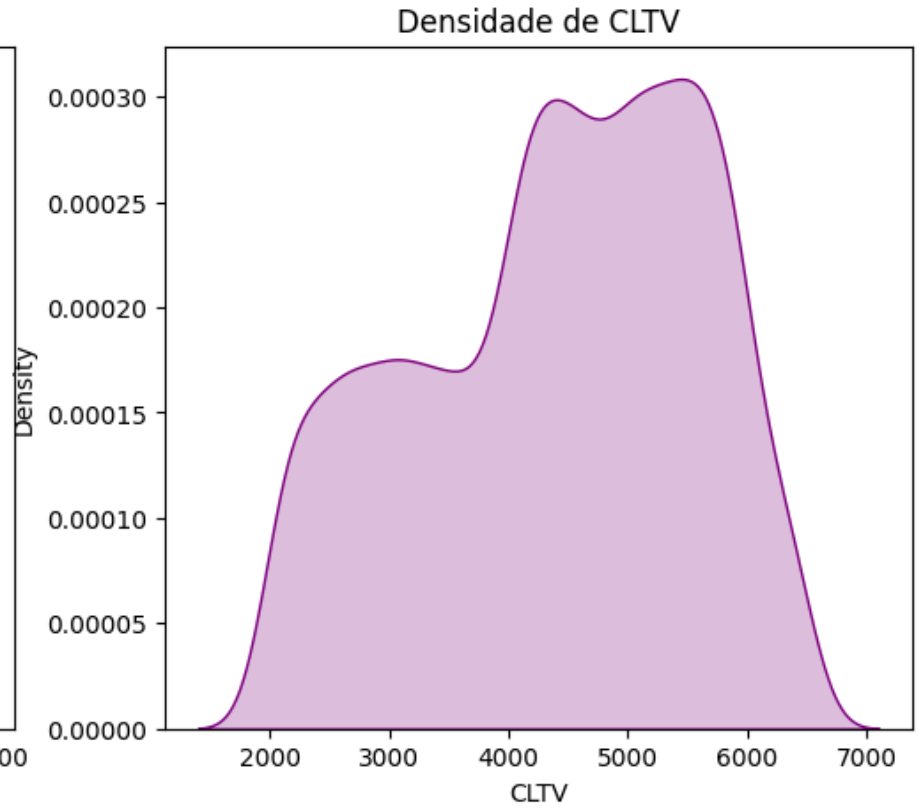
In [13]:
```
# Analisando as métricas principais
print(df_clean[['Tenure Months', 'Monthly Charges', 'Total Charges', 'CLTV']].describe())
```

```
       Tenure Months  Monthly Charges  Total Charges         CLTV
count    7043.000000      7043.000000    7043.000000  7043.000000
mean       32.371149        64.761692    2279.734304  4400.295755
std        24.559481        30.090047    2266.794470  1183.057152
min         0.000000        18.250000       0.000000  2003.000000
25%         9.000000        35.500000     398.550000  3469.000000
50%        29.000000        70.350000    1394.550000  4527.000000
75%        55.000000        89.850000    3786.600000  5380.500000
max        72.000000       118.750000    8684.800000  6500.000000
```

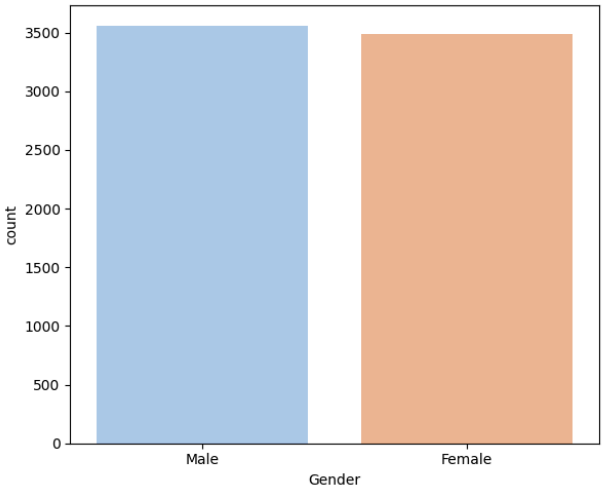
In [14]:

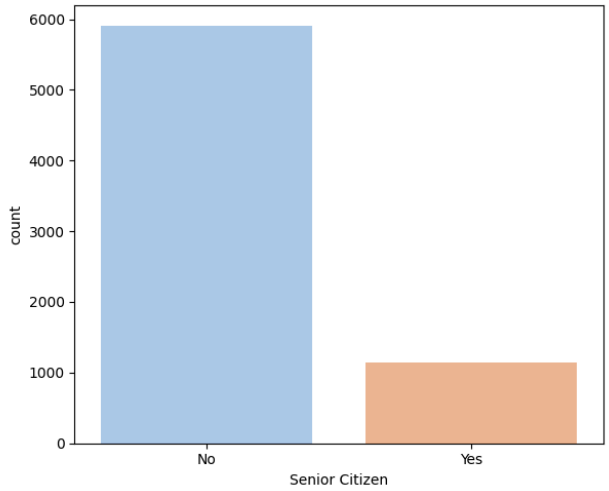
```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.kdeplot(df_clean['Total Charges'], fill=True, color="orange")
plt.title('Densidade de Total Charges')

plt.subplot(1, 2, 2)
sns.kdeplot(df_clean['CLTV'], fill=True, color="purple")
plt.title('Densidade de CLTV')

plt.show()
```

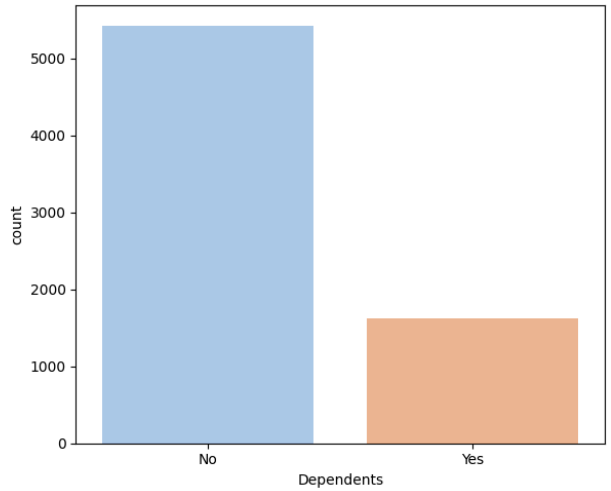




In [15]:
```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

sns.countplot(data=df_clean, x='Gender', hue='Gender', ax=axes[0], palette='pastel')
sns.countplot(data=df_clean, x='Senior Citizen', hue='Senior Citizen', ax=axes[1], palette='pastel')
sns.countplot(data=df_clean, x='Dependents', hue='Dependents', ax=axes[2], palette='pastel')

plt.tight_layout()
plt.show()
```

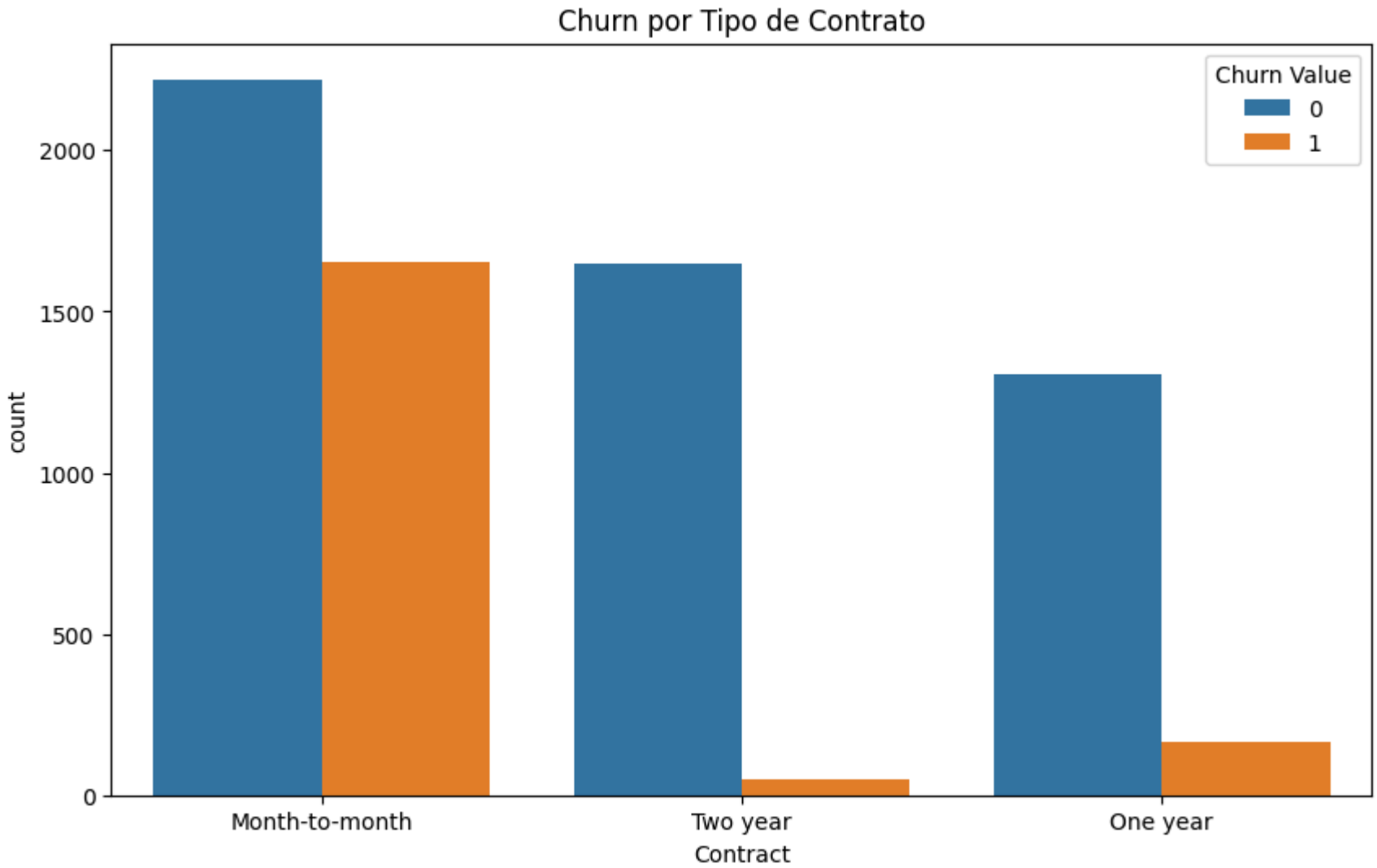




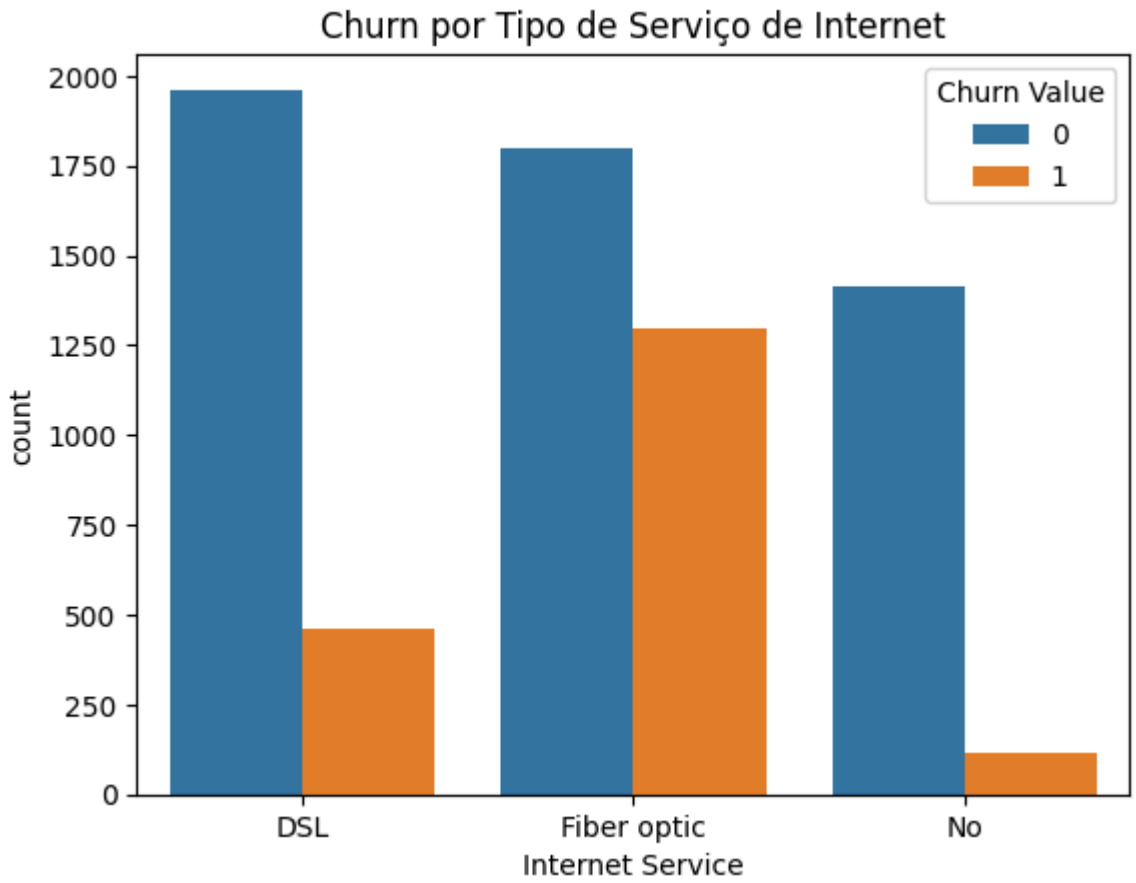


Análise Bivariada

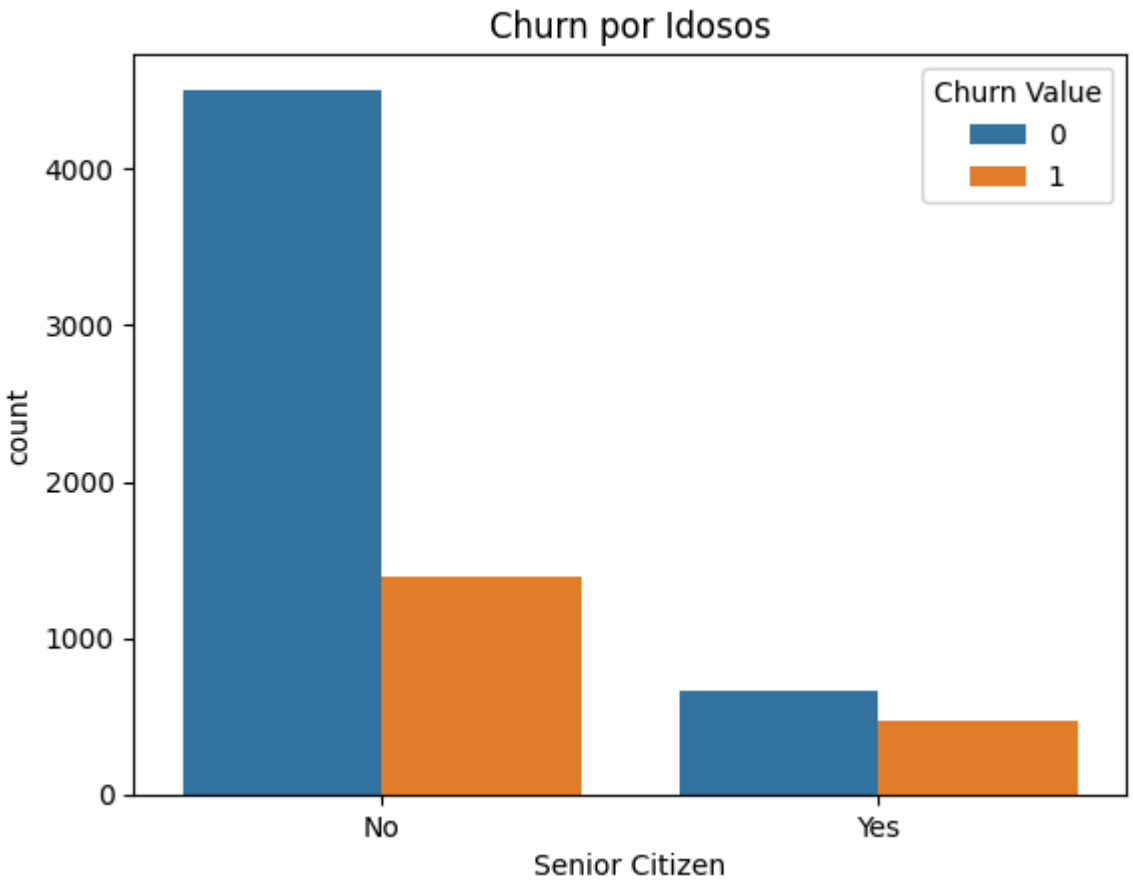
In [16]:
```
# Churn por Tipo de Contrato
plt.figure(figsize=(10,6))
sns.countplot(data=df_clean, x='Contract', hue='Churn Value')
plt.title('Churn por Tipo de Contrato')
plt.show()
```



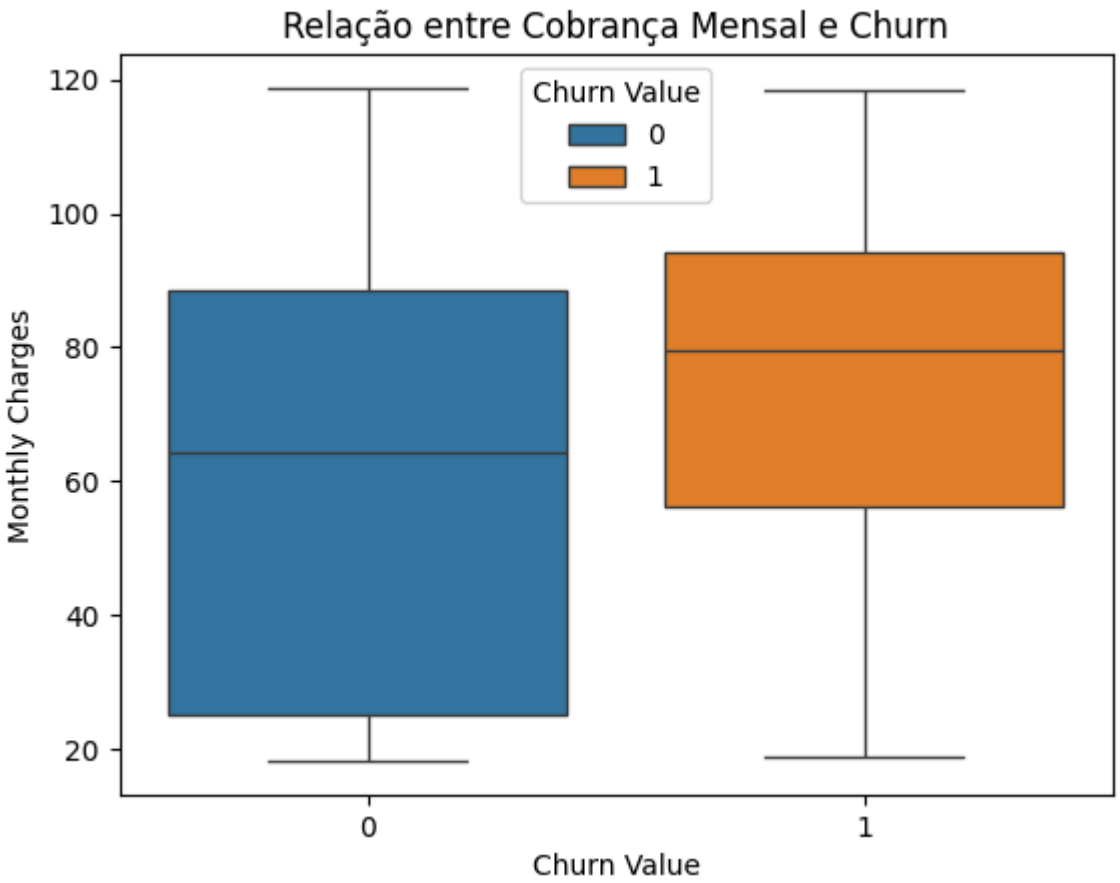
In [17]:
```
# Churn por Serviços de Internet
sns.countplot(data=df_clean, x='Internet Service', hue='Churn Value')
plt.title('Churn por Tipo de Serviço de Internet')
plt.show()
```



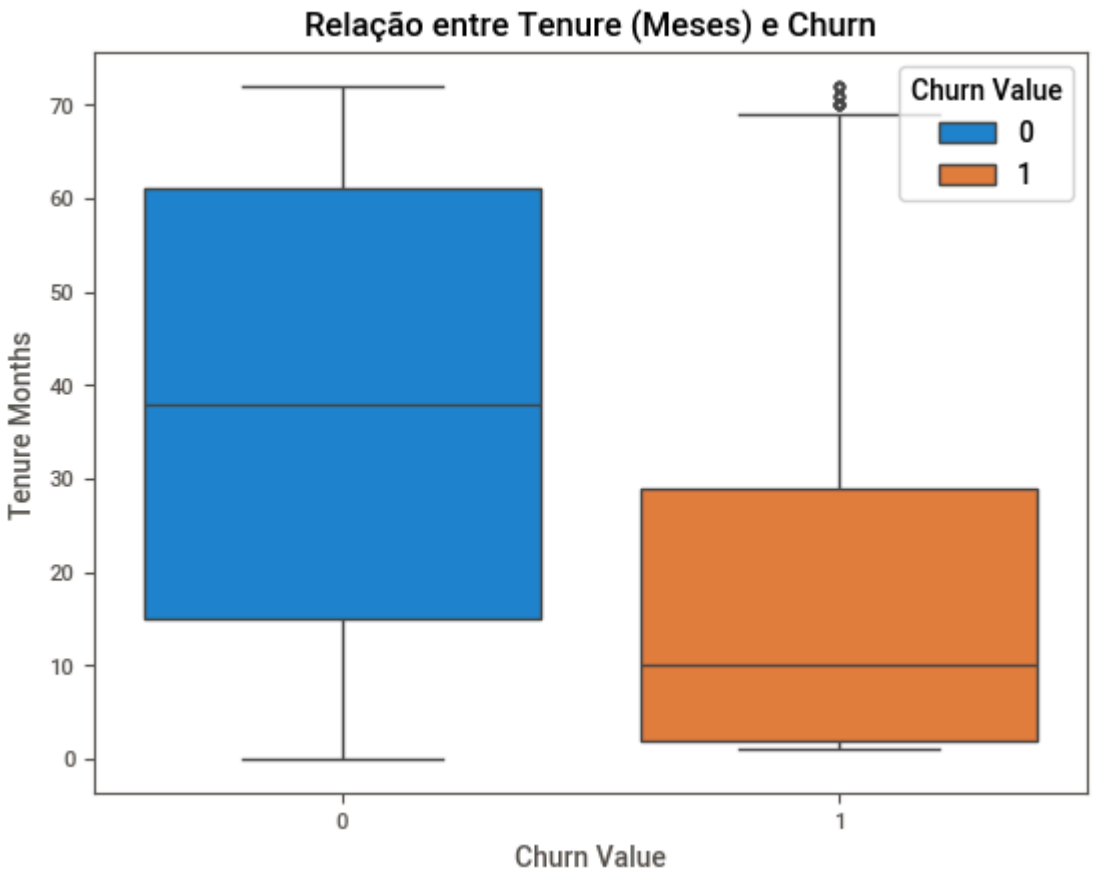
In [18]:
```
# Churn por Idosos
sns.countplot(data=df_clean, x='Senior Citizen', hue='Churn Value')
plt.title('Churn por Idosos')
plt.show()
```



In [19]:
```
# Boxplot: Mensalidades vs Churn
sns.boxplot(data=df_clean, x='Churn Value', y='Monthly Charges', hue='Churn Value')
plt.title('Relação entre Cobrança Mensal e Churn')
plt.show()
```

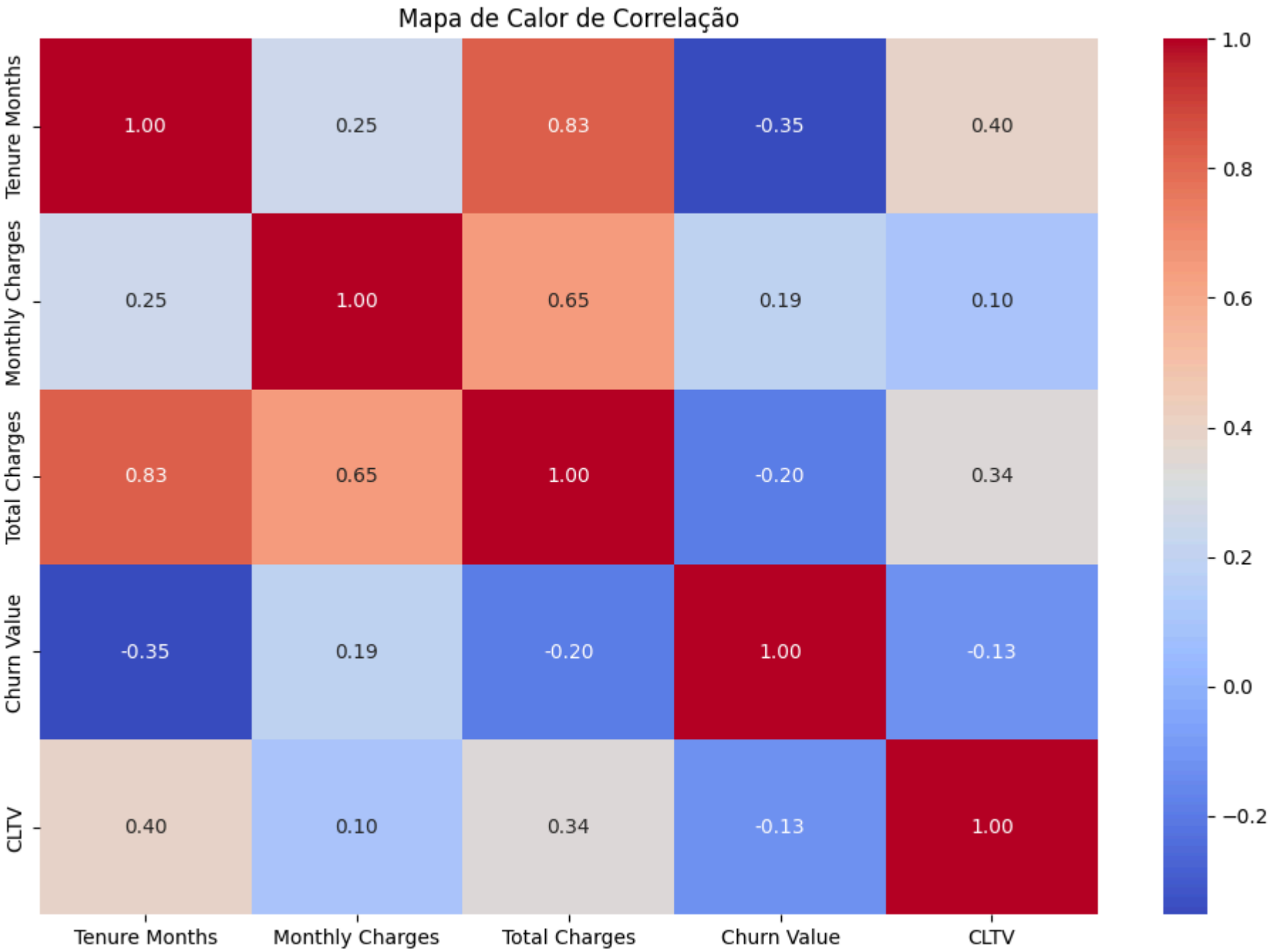


In [27]:
```
# Boxplot: Mensalidades vs Churn
sns.boxplot(data=df_clean, x='Churn Value', y='Tenure Months', hue='Churn Value')
plt.title('Relação entre Tenure (Meses) e Churn')
plt.show()
```

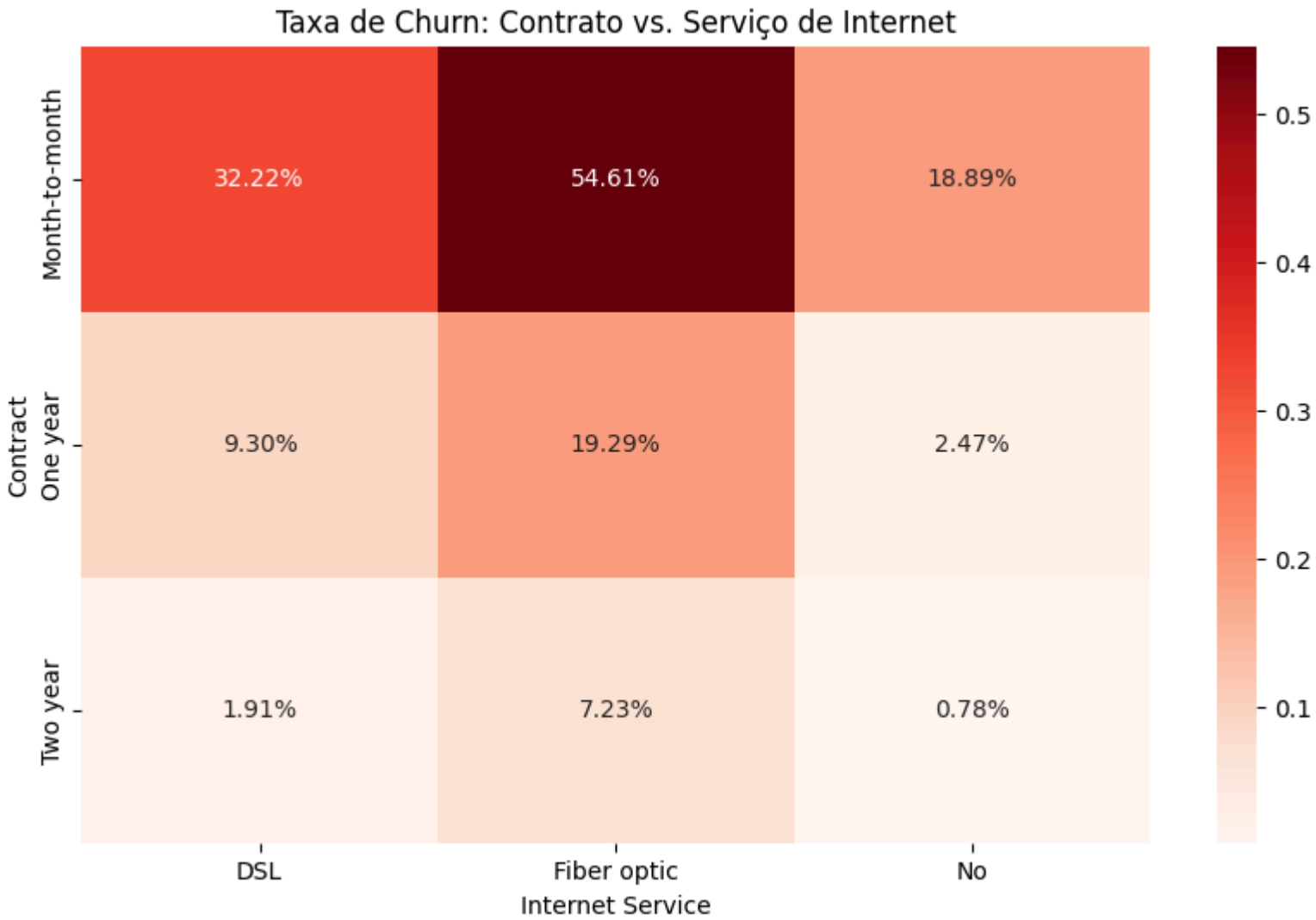


Análise Multivariada e Correlações

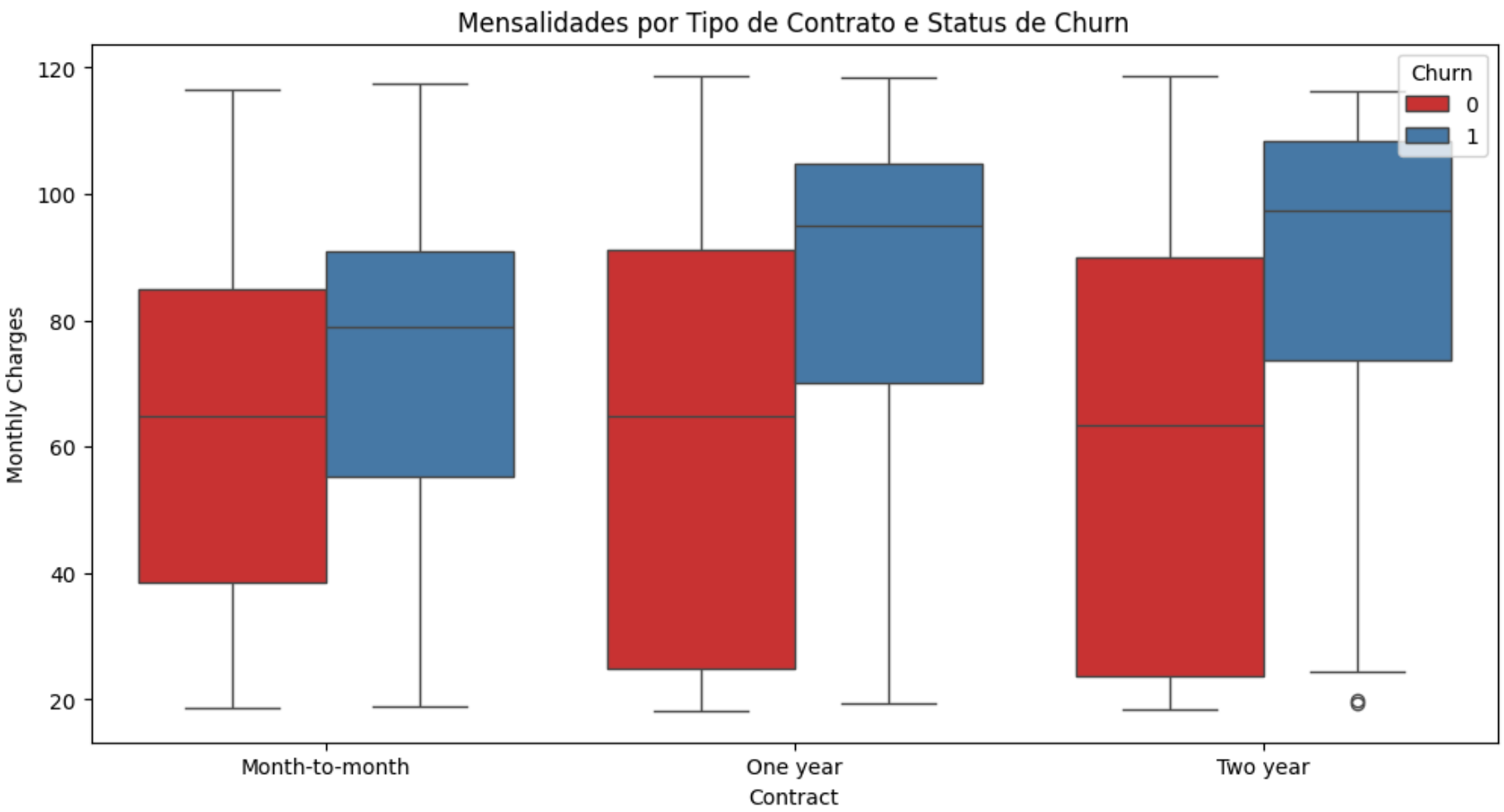
In [21]:
```
# Selecionando apenas colunas numéricas para a correlação
numeric_cols = df_clean.select_dtypes(include=['float64', 'int64']).columns
corr = df_clean[numeric_cols].corr()

plt.figure(figsize=(12, 8))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Mapa de Calor de Correlação')
plt.show()
```


Mapa de Calor de Correlação
Tenure Months
Monthly Charges
Total Charges
Churn Value
CLTV
1.00 0.25 0.83 -0.35 0.40
0.25 1.00 0.65 0.19 0.10
0.83 0.65 1.00 -0.20 0.34
-0.35 0.19 -0.20 1.00 -0.13
0.40 0.10 0.34 -0.13 1.00
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

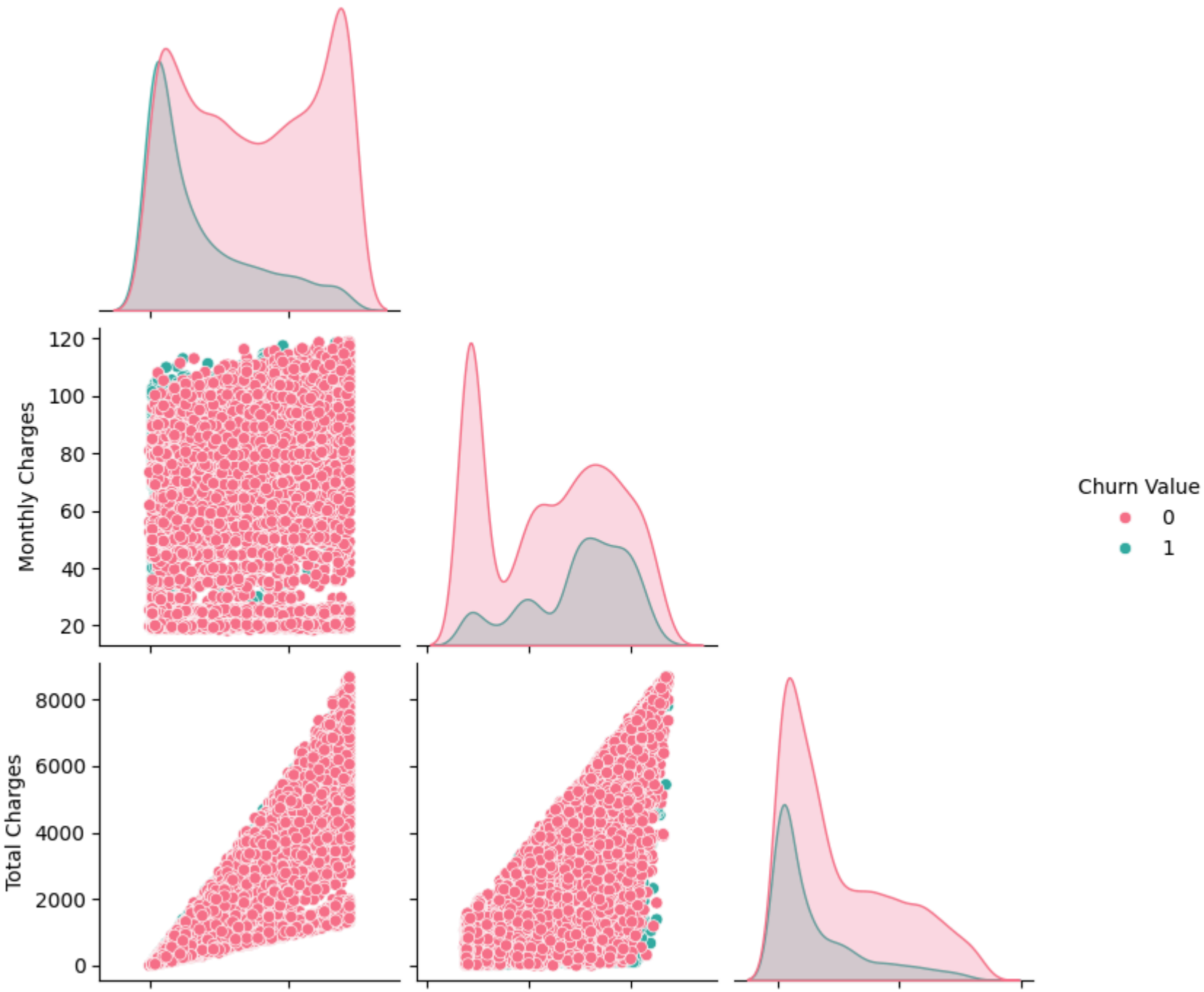
In [22]:
```
# Criando uma tabela dinâmica com a média de Churn (que representa a taxa)
pivot_churn = df_clean.pivot_table(index='Contract',
                                   columns='Internet Service',
                                   values='Churn Value',
                                   aggfunc='mean')

plt.figure(figsize=(10, 6))
sns.heatmap(pivot_churn, annot=True, cmap='Reds', fmt=".2%")
plt.title('Taxa de Churn: Contrato vs. Serviço de Internet')
plt.show()
```

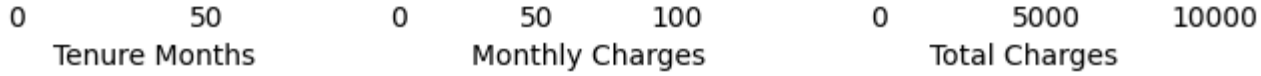


In [23]:
```
plt.figure(figsize=(12, 6))
sns.boxplot(data=df_clean, x='Contract', y='Monthly Charges', hue='Churn Value',
            order=['Month-to-month', 'One year', 'Two year'], palette='Set1')

plt.title('Mensalidades por Tipo de Contrato e Status de Churn')
plt.legend(title='Churn', loc='upper right')
plt.show()
```



In [24]:
```
# Selecionando apenas algumas colunas chave para não travar o computador
cols_key = ['Tenure Months', 'Monthly Charges', 'Total Charges', 'Churn Value']
sns.pairplot(df_clean[cols_key], hue='Churn Value', corner=True, palette='husl')
plt.show()
```


Monthly Charges
120
100
80
60
40
20
Total Charges
8000
6000
4000
2000
0
Churn Value
0
1



Eda automático

In [ ]:
```
# Opção 1
import sweetviz as sv

report = sv.analyze(df_clean, target_feat='Churn Value')
report.show_html('eda_telco_sweetviz.html', open_browser=False)
```

```
                                          |          | [  0%]   00:00 -> (? left)
Report eda_telco_sweetviz.html was generated.
```

In [30]:
```
# Opção 2
from ydata_profiling import ProfileReport

profile = ProfileReport(df_clean, title="Relatório de Churn - Telco IBM", explorative=True)

# Salvar relatório
profile.to_file("eda_telco_ydata.html")
```

```
Summarize dataset:   0%|          | 0/5 [00:00<?, ?it/s]
100%|██████████| 22/22 [00:00<00:00, 633.48it/s]
Generate report structure:   0%|          | 0/1 [00:00<?, ?it/s]
Render HTML:   0%|          | 0/1 [00:00<?, ?it/s]
Export report to file:   0%|          | 0/1 [00:00<?, ?it/s]
```